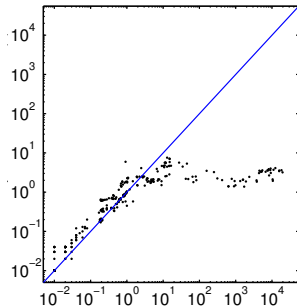


L08: Portfolio solvers, algorithm configuration and parallelism



Prof. Tias Guns and Dr. Hendrik 'Henk' Bierlee

KU LEUVEN

Partly based on slides from Lars Kotthoff and Toby Davies.

Lecture Overview

Modeling:

- ▶ L01: Introduction
- ▶ L02: Basic modeling
- ▶ L03: Global Constraints
- ▶ L04: Viewpoints and redundancy
- ▶ L05: Symmetry and dominance

Solving:

- ▶ L06: Solving technologies and encodings, part 1
- ▶ L07: Solving technologies and encodings, part 2
- ▶ **L08: Portfolio solvers, algorithm configuration and parallelism**

Advanced topics in CP research:

- ▶ L09: Topic TBA
- ▶ L10: Topic TBA

Portfolio solvers

Definition

A **portfolio solver** refers to a solver that has a *collection* of alternative search strategies, or even entire other solvers, that it chooses from.

Example portfolio solvers:

- ▶ In the Pseudo-Boolean competition, it has become common to first run an ILP solver for limited time, only then start the PB solver on the hard instances.
- ▶ Commercial ILP solvers include a local search solver to find good initial solutions and hence a good initial upper bound (for minimisation).
- ▶ Modern SAT solvers divide their time between 'focussed mode' (frequent restarts, good for finding SAT solutions) and 'stable mode' (very few restarts and different heuristics, good for proving UNSAT)

Portfolio solvers

Definition

A **portfolio solver** refers to a solver that has a *collection* of alternative search strategies, or even entire other solvers, that it chooses from.

Example portfolio solvers:

- ▶ In the Pseudo-Boolean competition, it has become common to first run an ILP solver for limited time, only then start the PB solver on the hard instances.
- ▶ Commercial ILP solvers include a local search solver to find good initial solutions and hence a good initial upper bound (for minimisation).
- ▶ Modern SAT solvers divide their time between 'focussed mode' (frequent restarts, good for finding SAT solutions) and 'stable mode' (very few restarts and different heuristics, good for proving UNSAT)

Portfolio solvers can contain **independent solvers**, meaning that their implementations are independent pieces of code, or **dependent solvers** that do share data structures or other information during search.

Portfolio solvers

A portfolio can be used in different ways:

- ▶ Choosing one solver from the portfolio to run: **algorithm selection**
- ▶ Choosing a sequence of time-limited solver runs: **portfolio sequencing/scheduling**
- ▶ Choosing which subset of solvers to run in parallel: **parallel portfolio selection**

Furthermore, this can be done statically (decided upfront, e.g. through **algorithm configuration**), or dynamically (choosing which/how long during search, based on the current state).

Portfolio solvers

A portfolio can be used in different ways:

- ▶ Choosing one solver from the portfolio to run: **algorithm selection**
- ▶ Choosing a sequence of time-limited solver runs: **portfolio sequencing/scheduling**
- ▶ Choosing which subset of solvers to run in parallel: **parallel portfolio selection**

Furthermore, this can be done statically (decided upfront, e.g. through **algorithm configuration**), or dynamically (choosing which/how long during search, based on the current state).

CPMpy is NOT a portfolio solver, because it does not automatically choose which solver to run for you (it defaults to ortools). But you *could* build a portfolio solver on top of CPMpy that would choose a solver intelligently.

Outline

1. Algorithm Selection
2. Algorithm Configuration
3. Parallel portfolios
4. Other parallel paradigms

Algorithm Selection and Configuration

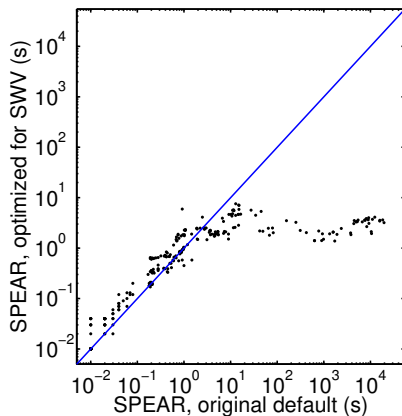
Which solver from the portfolio to use? Which choices to make?

Algorithm Selection choose the best *algorithm* for solving a problem

Algorithm Configuration choose the best *parameter configuration*,
even for a single solver

Large performance improvements possible, without changing the model!

Performance Improvements example



Hutter, Frank, Domagoj Babic, Holger H. Hoos, and Alan J. Hu. "Boosting Verification by Automatic Tuning of Decision Procedures." FMCAD 07.

Algorithm Selection

Given a problem, choose the best algorithm to solve it.

“algorithm” used in a very loose sense

- ▶ constraint solvers
- ▶ search strategies
- ▶ modelling choices
- ▶ different types of consistency
- ▶ ...

As long as a finite (typically small) set of “algorithms” is available: the *portfolio*.

Rice, John R. “The Algorithm Selection Problem.” *Advances in Computers* 15 (1976): 65–118.

(Automatic) algorithm selection

Automatic algorithm selection focusses on 'instance specific' configuration:

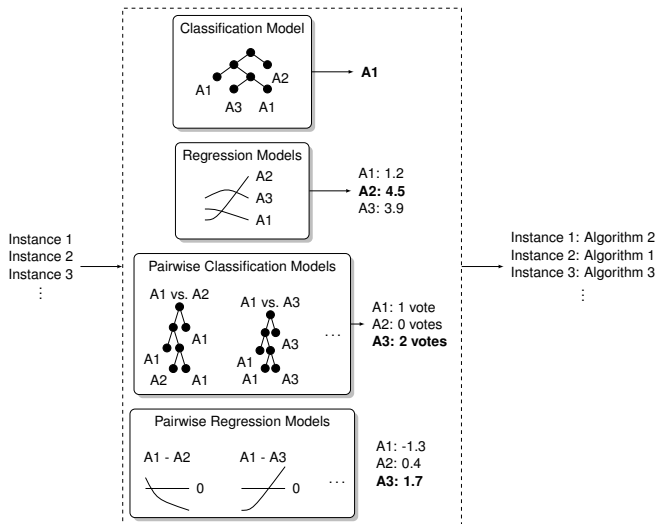
- ▶ For an unseen problem instance: which algorithm should I run on it?
- ▶ Instance-specific: allowed to analyse the instance and decide based on *features* that you can derive from it
 - ▶ nr of variables, constraints
 - ▶ types of constraints used
 - ▶ structure of the constraint network
 - ▶ probing (run an algorithm for short time, collect statistics)
- ▶ Portfolio solver developers often hard-code their experience into if/then rules

(Automatic) algorithm selection

Automatic algorithm selection focusses on 'instance specific' configuration:

- ▶ For an unseen problem instance: which algorithm should I run on it?
- ▶ Instance-specific: allowed to analyse the instance and decide based on *features* that you can derive from it
 - ▶ nr of variables, constraints
 - ▶ types of constraints used
 - ▶ structure of the constraint network
 - ▶ probing (run an algorithm for short time, collect statistics)
- ▶ Portfolio solver developers often hard-code their experience into if/then rules
- ▶ In *automatic* algorithm selection, Machine Learning is typically used to build a **performance model**, and predict which algorithm to use.
 - ▶ that means you need a *training set* of instances for the ML model
 - ▶ and a *test set* of instances on which to evaluate

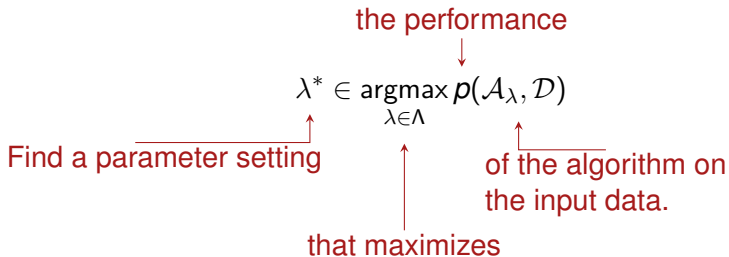
Types of Performance Models for automatic algorithm selection



Outline

1. Algorithm Selection
- 2. Algorithm Configuration**
3. Parallel portfolios
4. Other parallel paradigms

Algorithm Configuration = Automated Parameter Tuning



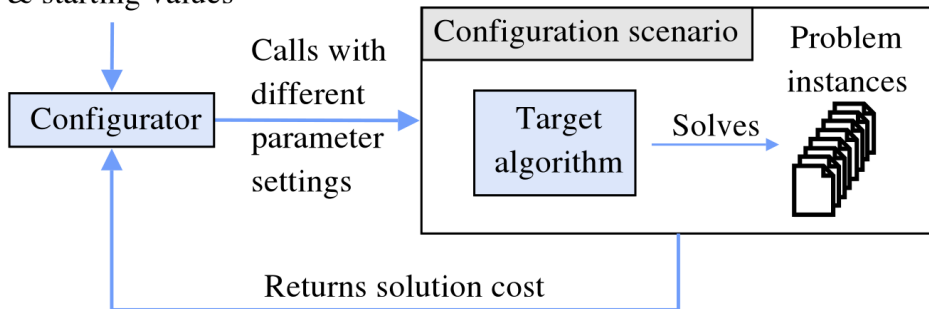
Parameters?

- ▶ anything you can change that makes sense to change
- ▶ typically multiple parameters with many values (possibly continuous): a **large** (possibly infinite) set
- ▶ e.g. search heuristic, variable ordering, type of global constraint decompositions, subsolvers to try and for how long, different thresholds, ...
- ▶ Example: CPLEX MIP solver, 76 parameters; Spear SAT solver, 26 parameters

Typically for a *set* of related instances (e.g. nurse rostering problems that the hospital solves every day)

Algorithm Configuration

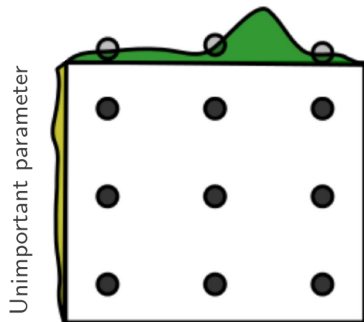
Parameter domains
& starting values



Frank Hutter and Marius Lindauer, "Algorithm Configuration: A Hands on Tutorial", AAAI 2016

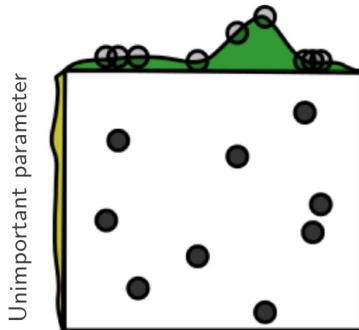
Grid and Random Search

Grid Layout



Important parameter

Random Layout



Important parameter

Bergstra, James, and Yoshua Bengio. "Random Search for Hyper-Parameter Optimization." J. Mach. Learn. Res. 13, no. 1 (February 2012): 281–305.

Automatic algorithm configuration

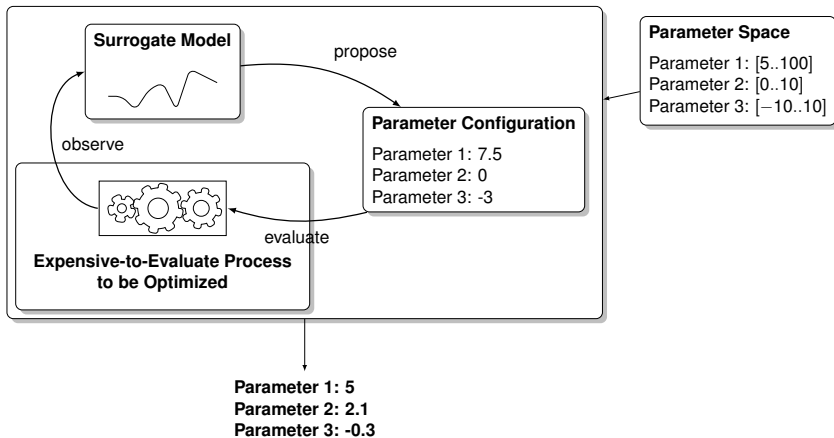
Can we do better than grid/random?

Target exploration to a limited number of promising configurations, within a time budget.

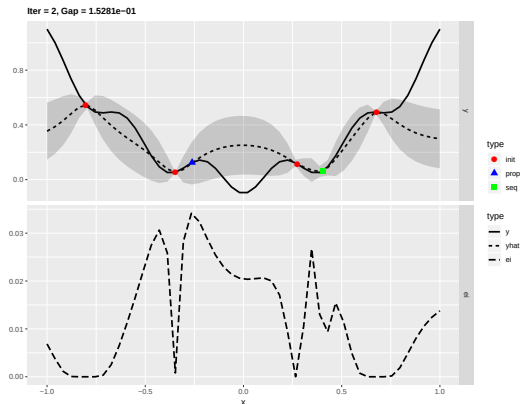
Example: **model-based optimisation** (SMBO)

- ▶ evaluate a small number of initial (random) configurations
- ▶ use (probabilistic) Machine Learning to train a *surrogate model* of parameter-performance surface based on this
- ▶ use an *acquisition function* to decide most promising configuration to try next
- ▶ repeat, stop when resources exhausted or desired solution quality achieved

Model-Based Optimization (black-box optimisation)

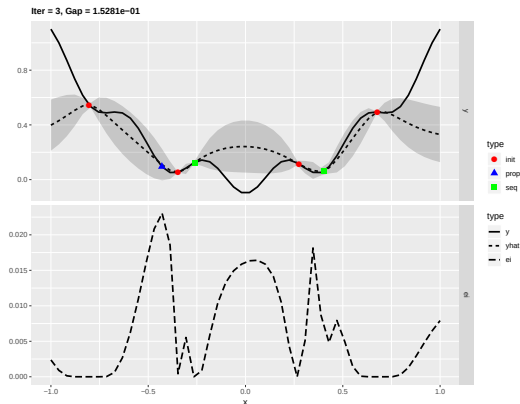


Sequential Model-Based Optimization (SMBO) with Gaussian Processes



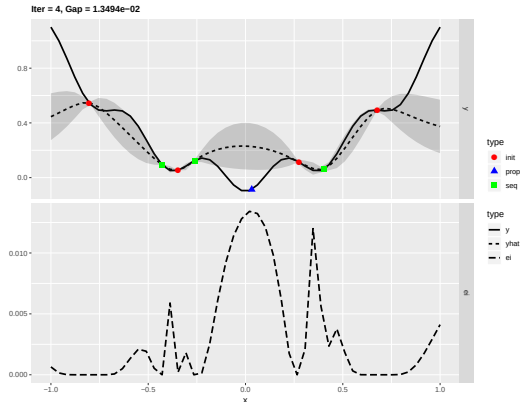
Bischl, Bernd, Jakob Richter, Jakob Bossek, Daniel Horn, Janek Thomas, and Michel Lang.
“MlrMBO: A Modular Framework for Model-Based Optimization of Expensive Black-Box Functions,”

Sequential Model-Based Optimization (SMBO) with Gaussian Processes



Bischl, Bernd, Jakob Richter, Jakob Bossek, Daniel Horn, Janek Thomas, and Michel Lang.
“MlrMBO: A Modular Framework for Model-Based Optimization of Expensive Black-Box Functions,”

Sequential Model-Based Optimization (SMBO) with Gaussian Processes



Bischl, Bernd, Jakob Richter, Jakob Bossek, Daniel Horn, Janek Thomas, and Michel Lang.
“MlrMBO: A Modular Framework for Model-Based Optimization of Expensive Black-Box Functions,”

Time Budget

How much time/how many function evaluations to give to the configurator?

- ▶ too much → wasted resources
- ▶ too little → suboptimal result
- ▶ use statistical tests?
- ▶ evaluate on parts of the instance set
- ▶ for runtime: adaptive capping = solver timeout with current best runtime.

Need to evaluate on an unseen test-set to avoid 'over-tuning'.

Time Budget

How much time/how many function evaluations to give to the configurator?

- ▶ too much → wasted resources
- ▶ too little → suboptimal result
- ▶ use statistical tests?
- ▶ evaluate on parts of the instance set
- ▶ for runtime: adaptive capping = solver timeout with current best runtime.

Need to evaluate on an unseen test-set to avoid 'over-tuning'.

There exists a number of out-of-the-box libraries you can use for parameter configuration (*hyperopt*, *smac3*, ...)

Solver configuration options

These are often very solver-specific!!

OrTools CP-SAT:

- ▶ "cp_model_probing_level": how much effort on probing
- ▶ "search_branching": search strategy (automatic, fixed, hint, LP-based, ...)
- ▶ "symmetry_level": symmetry detection and breaking before/during search
- ▶ "use_phase_saving": value selection = last value the variable was assigned to

Choco: In Python API only some search strategies are exposed.

Gurobi categories:

- ▶ Algorithmic control parameters for presolve, simplex, cuts, ...
- ▶ Tolerance parameters which control solution quality requirements (not to be tuned automatically)

Outline

1. Algorithm Selection
2. Algorithm Configuration
- 3. Parallel portfolios**
4. Other parallel paradigms

Parallel portfolios

Almost all computers nowadays have multiple cores.

Why use only one?

Parallel portfolios

Almost all computers nowadays have multiple cores.

Why use only one?

Idea: try many varied search strategies in parallel.

Some worker is likely to get lucky and find a good solution!

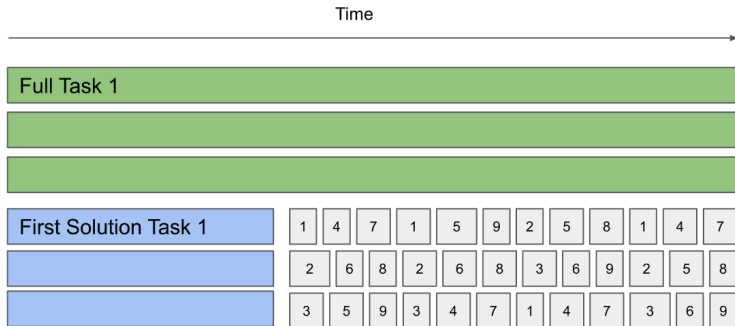
- ▶ *Diversity* in strategies can be more important than individual strength

OrTools CP-SAT parallel portfolio deep-dive

OrTools CP-SAT is very much a (parallel) portfolio solver (T. Davies, 2025):

- ▶ **18 full problem subsolvers:** [core, default_lp, fixed, lb_tree_search, max_lp, no_lp, objective_lb_search, objective_lb_search_max_lp, objective_lb_search_no_lp, objective_shaving_max_lp, objective_shaving_no_lp, probing, probing_max_lp, probing_no_lp, pseudo_costs, quick_restart, quick_restart_no_lp, reduced_costs]
- ▶ **46 first solution subsolvers:** [fj(14), fj_lin(6), fs_random(7), fs_random_no_lp(7), fs_random_quick_restart(6), fs_random_quick_restart_no_lp(6)]
- ▶ **31 interleaved subsolvers:** [feasibility_pump, graph_arc_ins, graph_cst_ins, graph_dec_ins, graph_var_ins, lb_relax_ins, ls(11), ls_lin(5), rins/rens, rnd_cst_ins, rnd_var_ins, routing_full_path_ins, routing_path_ins, routing_random_ins, variables_shaving, variables_shaving_max_lp, variables_shaving_no_lp]

OrTools CP-SAT parallel portfolio deep-dive



For n workers:

- ▶ Start $n/2$ full problem solvers
- ▶ Start $n/2$ first solution subsolvers. When one has finished, continue interleaving (heuristic) subsolvers in that thread.

OrTools CP-SAT parallel portfolio deep-dive

This many independent solvers would be wasteful...

What they share:

- ▶ Bounds on the objective function
- ▶ The best 3 solutions found so far
- ▶ Root-level variable bounds (=unary constraints)
- ▶ Binary clauses
- ▶ Some short learned (conflict analysis) clauses

OrTools CP-SAT parallel portfolio deep-dive

This many independent solvers would be wasteful...

What they share:

- ▶ Bounds on the objective function
- ▶ The best 3 solutions found so far
- ▶ Root-level variable bounds (=unary constraints)
- ▶ Binary clauses
- ▶ Some short learned (conflict analysis) clauses

Can not share all learned clauses: more sharing is slower sharing, more clauses can lead to slower memory access.

Outline

1. Algorithm Selection
2. Algorithm Configuration
3. Parallel portfolios
4. Other parallel paradigms

Other parallel paradigms

A parallel portfolio is one parallel paradigm, very good up to a reasonable number of cores (16-24).

What if you want to speed up across *many many* cores? 64+?
parallel exploration of a shared search tree!

Two generic paradigms:

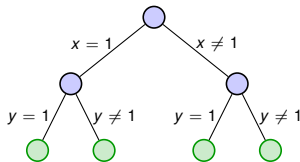
- ▶ Embarrassingly parallel search
- ▶ Work stealing

Embarrassingly parallel search

If you can split a problem into *independent* sub-problems, then it becomes (embarrassingly) easy to solve it in parallel.

For search trees: do a breadth-first enumeration up to depth n .

E.g. a binary search tree would create 2^n independent subtrees:



Each set of choices up to that depth, when added as a constraint, creates an independent subproblem: the best solution across all is the best for the original problem.

Embarrassingly parallel search

If you can split a problem into *independent* sub-problems, then it becomes (embarrassingly) easy to solve it in parallel.

Pro:

- ▶ Easy and fast to do the splitting
- ▶ No interaction between subproblems needed

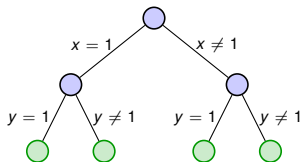
Con:

- ▶ Workers have to wait for the subproblem splitting to be ready (could do depth-limited DFS and a queue)
- ▶ Subproblems can be very unbalanced: some might be trivial to solve, others take as much time as the original problem
- ▶ Workers become idle when the queue is empty

Work stealing

Give the search tree to one worker.

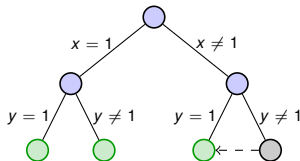
Until finished: if a worker has no subtree, *steal* part of the search tree of another worker.



Work stealing

Give the search tree to one worker.

Until finished: if a worker has no subtree, *steal* part of the search tree of another worker.

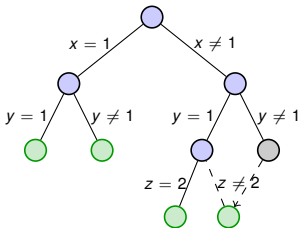


Requires the worker with the tree to spend time *splitting* its search tree and communicating it to the idle worker.

Work stealing

Give the search tree to one worker.

Until finished: if a worker has no subtree, *steal* part of the search tree of another worker.



Requires the worker with the tree to spend time *splitting* its search tree and communicating it to the idle worker.

Work stealing

Give the search tree to one worker.

Until finished: if a worker has no subtree, *steal* part of the search tree of another worker.

Pro:

- ▶ Workers are never idle: the search tree can always be split further

Con:

- ▶ Splitting and its communication needs modified implementations (e.g. split where, and make sure the original worker does not backtrack over it)
- ▶ Splitting takes time and does not involve actually searching
- ▶ Subproblems can be unbalanced, leading idle workers to keep interrupting the busy workers (e.g. with 40+ workers)

Summary

Single-thread portfolio:

Algorithm Selection choose the best *algorithm* for solving a problem

Algorithm Configuration choose the best *parameter configuration* for solving a problem with an algorithm

Parallelism:

- ▶ OrTools CP-SAT as example of a **parallel portfolio**: full workers, first solution workers and interleaved heuristic workers.
- ▶ Embarrassingly parallel search tree splitting
- ▶ Work stealing (ad-hoc search tree splitting)